

Replication Guide

This document provides step-by-step instructions to exactly reproduce all experimental results reported in the accompanying paper (v1–v7 all versions + extended experiments).

Prerequisites

- Python ≥ 3.10
- pip package manager
- Operating system: Linux, macOS, or Windows

Step 1: Clone the Repository

```
git clone https://github.com/yelaiyuluo/ISE_Coursework_TairuiZhang.git
cd ISE_Coursework_TairuiZhang
```

Step 2: Install Dependencies

```
pip install -r requirements.txt
```

Exact versions used in our experiments:

Package	Version	Required For
numpy	2.2.5	All versions
pandas	3.0.1	All versions
scikit-learn	1.8.0	All versions
scipy	1.15.2	All versions
optuna	4.3.0	Supplementary & extended experiments (TPE only)
lightgbm	$\geq 4.0.0$	v3 experiment only
matplotlib	$\geq 3.7.0$	Figure generation only

Step 3: Verify Datasets

Ensure the datasets/ directory contains the following 8 CSV files:

File	Rows	Features
7z.csv	68,640	8
Apache.csv	640	8
brotli.csv	180	2
LLVM.csv	65,536	16
PostgreSQL.csv	864	8
spear.csv	16,384	14
storm.csv	1,557	12
x264.csv	4,608	10

Step 4: Run v1 Main Experiment (FLASH-RF vs Random Search)

```
cd iteration1
python flash_tuner.py --datasets ../datasets --budget 100 --repeats 30 --output results
```

- 30 independent runs of Random Search and FLASH-RF v1 on each of 8 datasets
- **Expected runtime:** approximately 30–60 seconds

Output:

```
results/
├── summary.csv          # Summary table (corresponds to Table 1)
├── results.json         # Full results with raw data
└── raw/                # Per-system raw results
```

Step 5: Run Supplementary Experiments (TPE Comparison + Sensitivity)

```
cd iteration1
python run_ablation.py --datasets ../datasets \
    --budget 100 --repeats 30 --output results/ablation
```

- **Experiment 1 (TPE comparison):** RS, FLASH-RF, TPE three-way comparison
- **Experiment 2 (Sensitivity):** One-factor-at-a-time sweeps of `init_ratio`, `batch_size`, `exploit_ratio`
- **Expected runtime:** approximately 20–25 minutes

Step 6: Run v2 Experiment (RF-LCB + Feature Preprocessing)

```
cd iteration2
PYTHONPATH=../iteration1 python flash_tuner_v2.py \
    --datasets ../datasets --budget 100 --repeats 30 --output results
```

- Compares RS, v1, v2, and TPE
- **Expected runtime:** approximately 3–5 minutes

Step 7: Run v3 Experiment (LightGBM Ensemble, Rejected)

Additional dependency: `pip install lightgbm` (macOS also requires `brew install libomp`)

```
cd iteration3
PYTHONPATH=../iteration1:../iteration2 \
    python flash_tuner_v3.py \
    --datasets ../datasets --budget 100 --repeats 30 --output results
```

- Compares RS, v2, and v3-LGB
- **Expected runtime:** approximately 5–8 minutes

Step 8: Run v4 Experiment (RF + Thompson Sampling)

```
cd iteration4
PYTHONPATH=../iteration1:../iteration2 \
    python flash_tuner_v4.py \
    --datasets ../datasets --budget 100 --repeats 30 --output results
```

- Compares RS, v2, and v4
- **Expected runtime:** approximately 4–5 minutes

Step 9: Run v5 Experiment (RF + TS + Conditional Log Transform)

```
cd iteration5
PYTHONPATH=../iteration1:../iteration2:../iteration4 \
python flash_tuner_v5.py \
--datasets ../datasets --budget 100 --repeats 30 --output results
```

- Compares RS, v4, and v5
- **Expected runtime:** approximately 3–4 minutes

Step 10: Run v6 Experiment (v5 + Conditional Stratified Sampling, Final Version)

```
cd iteration6
PYTHONPATH=../iteration1:../iteration2:../iteration4:../iteration5 \
python flash_tuner_v6.py \
--datasets ../datasets --budget 100 --repeats 30 --output results
```

- Compares RS, v5, and v6
- **Expected runtime:** approximately 3–4 minutes

Step 11: Run v7 Experiment (Conditional Rank Transform, Rejected)

```
cd iteration7
PYTHONPATH=../iteration1:../iteration2:../iteration4:../iteration5:../iteration6 \
python flash_tuner_v7.py \
--datasets ../datasets --budget 100 --repeats 30 --output results_v7
```

- Compares RS, v6, and v7
- **Expected runtime:** approximately 3–4 minutes
- **Expected result:** v7 degrades on 3 systems (7z, spear, storm), total hit-optimal drops from 181 to 175, confirming v6 as the final version

Step 12: Run Extended Experiments (6-Method Full Comparison × Multi-Budget)

```
cd iteration6
PYTHONPATH=../iteration1:../iteration2:../iteration4:../iteration5 \
python run_extended_experiments.py \
--datasets ../datasets --output results_extended
```

- **6 methods:** RS, HC, SA, BestConfig, TPE, FLASH-RF v6
- **3 budgets:** R=30, 100, 200
- **8 systems × 30 repeats**
- **Expected runtime:** approximately 10–15 minutes

Step 13: Generate Report Figures

```
python docs/generate_figures.py
```

- Generates 4 report figures to docs/figures/
- **Expected runtime:** < 10 seconds

Verifying Results

Compare each step’s summary.csv with the corresponding table in the report. Results should match exactly because all algorithms are fully deterministic given the same seed.

Note: All versions use the same unified seed sequence $s_i = 42i + 1$ ($i = 0, \dots, 29$), ensuring fair cross-version comparison and reproducibility.

Version Quick Reference

Ver.	Script (iteration*/)	Core Algorithm	Status	§
v1	flash_tuner.py	RF(10) + mean ranking	Baseline	2
v2	flash_tuner_v2.py	RF(20) + LCB + feature preprocessing	Comparison	3
v3	flash_tuner_v3.py	LightGBM ensemble + LCB	Rejected	4
v4	flash_tuner_v4.py	RF(20) + Thompson Sampling	Comparison	5
v5	flash_tuner_v5.py	RF(20) + TS + conditional log transform	Comparison	6.5
v6	flash_tuner_v6.py	v5 + conditional stratified sampling	Final	6.6
v7	flash_tuner_v7.py	v6 + conditional rank transform	Rejected	6.7

Reproducibility Notes

1. **Deterministic seeds:** All versions use the unified seed sequence $s_i = 42i + 1$ ($i = 0, \dots, 29$), ensuring identical results across executions and fair cross-version comparison.
2. **Budget accounting:** Valid configurations consume 1 unit of budget; invalid configurations do not consume budget.
3. **All systems use minimisation:** Lower performance values are better.
4. **Statistical test:** One-sided Mann–Whitney U test with H_1 : method produces lower (= better) performance values than baseline, $\alpha = 0.05$.
5. **TPE experiment note:** TPE is implemented via Optuna’s TPESampler. When TPE produces 40 consecutive invalid / duplicate suggestions, it falls back to random sampling from unmeasured valid configurations.
6. **v3 LightGBM dependency:** On macOS, LightGBM requires libomp (install via `brew install libomp`).
7. **Cross-platform consistency:** NumPy’s RandomState ensures cross-platform reproducibility.